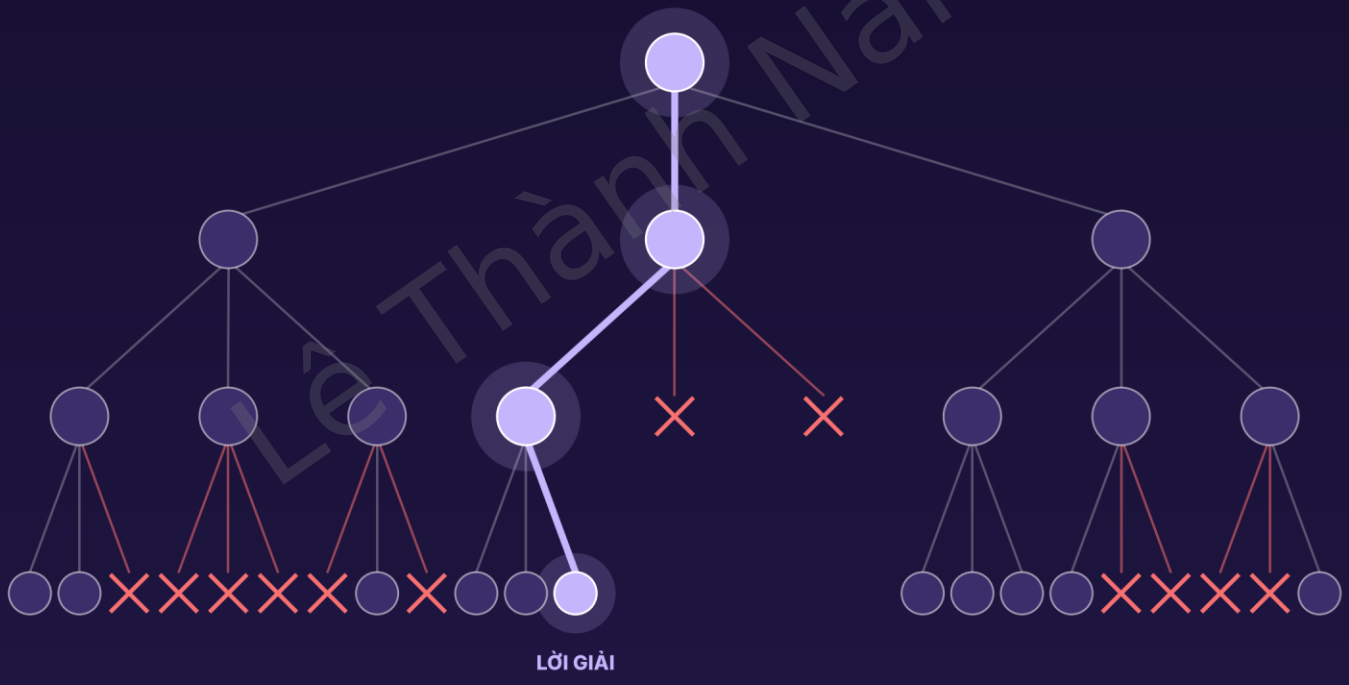
TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

ĐỆ QUY
QUAY LUI
và NHÁNH CẬN

TRY · RECURSE · UNDO · PRUNE

TẬP 1 · LÝ THUYẾT

Lê Thành Nam



01 Chương 1. ĐỆ QUY
02 Chương 2. QUAY LUI
03 Chương 3. NHÁNH CẬN
04 Chương 4. KĨ THUẬT NÂNG CAO

*12 bài học · 4 chương · từ hàm đệ quy đến tìm kiếm sâu dần IDA**

Dành cho học sinh THPT ôn luyện học sinh giỏi cấp tỉnh và tiếp cận cấp quốc gia

Tổng hợp và biên soạn: Lê Thành Nam

LỜI NÓI ĐẦU

Có những bài toán mà ta không biết công thức, không thấy quy luật, cũng chẳng có thuật toán tham lam nào đúng: xếp tám quân hậu lên bàn cờ để không quân nào ăn được quân nào, điền một ô Sudoku, tìm hành trình ngắn nhất đi qua mười lăm thành phố, chọn những món đồ đáng giá nhất cho vào ba lô. Với những bài toán như vậy, con đường chắc chắn nhất là **thử**: thử một lựa chọn, đi tiếp; nếu đi vào ngõ cụt thì **quay lui** và thử lựa chọn khác. Đó là phương pháp **quay lui** (backtracking), và nền móng của nó là **đệ quy**: một hàm tự gọi chính nó.

Nhưng "thử hết" thường có nghĩa là thử hàng tỉ tỉ khả năng. Nghệ thuật thật sự nằm ở chỗ **không thử những gì chắc chắn vô ích**. Một nhánh đã vi phạm ràng buộc thì cắt ngay; một nhánh dù đi tốt đến đâu cũng không thể hơn lời giải đã có thì cũng cắt luôn. Đó là kĩ thuật **nhánh cận** (branch and bound). Với cùng một bài toán, chương trình quay lui ngây thơ có thể chạy hàng giờ, còn chương trình có cắt nhánh tốt chỉ cần một phần mười giây.

Trong đề thi học sinh giỏi, quay lui và nhánh cận có mặt ở cả hai đầu của thang điểm. Ở đầu dưới, chương trình vét cạn đúng đắn là cách chắc chắn nhất để giành trọn **subtask đầu tiên** của gần như mọi bài, và là "trâu" để đối chiếu khi viết lời giải nhanh. Ở đầu trên, những bài có không gian tìm kiếm khổng lồ nhưng có cấu trúc (n quân hậu với n lớn, cái túi với trọng lượng tới 10^9, người du lịch, Sudoku khó) chỉ chịu thua trước một chương trình **cắt nhánh khéo léo**.

Bộ tài liệu này gồm ba tập đi cùng nhau:

- **Tập 1: Lý thuyết** (cuốn sách này) gồm 12 bài học chia thành 4 chương: Đệ quy; Quay lui; Nhánh cận; Kĩ thuật nâng cao (thứ tự duyệt, mặt nạ bit, ghi nhớ trạng thái, tìm kiếm sâu dần).
- **Tập 2: Bài tập** gồm 36 bài, mỗi bài học có ba bài tập ở ba mức độ (cơ bản, trung bình, nâng cao), có ràng buộc theo subtask như đề thi thật.
- **Tập 3: Hướng dẫn giải** phân tích từng bài tập từ cách vét cạn ngây thơ đến cách cắt nhánh tốt, kèm chương trình hoàn chỉnh và bảng điểm thực tế của từng cách làm.

Mỗi bài học không đưa ra thuật toán ngay. Em sẽ được đặt trước một bài toán cụ thể, thử cách ngây thơ, **đếm số trạng thái** để thấy chỗ bế tắc, rồi được dẫn từng bước để phá bế tắc. Mọi cây tìm kiếm đều được vẽ ra và chạy tay; mọi con số về thời gian chạy và số nút được thăm trong sách đều được **đo thật**, không phải ước lượng; mọi chương trình đều đã được kiểm tra bằng cách đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường, giống như cách một người giỏi thật sự suy nghĩ khi gặp bài mới:

Phần	Nội dung
Khởi động	Một câu hỏi nhỏ, gần gũi để em bắt đầu nghĩ.
Bài toán	Đề bài cụ thể, có ví dụ và hình minh họa.
Ý tưởng ngây thơ	Cách làm đầu tiên ai cũng nghĩ ra, kèm phần đếm số trạng thái phải thử.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ chậm, và đặt câu hỏi dẫn tới ý tưởng mới.
Phá bế tắc	Từng bước: quan sát, cây tìm kiếm, cách cắt nhánh, chạy tay, chứng minh, chương trình.
Mở rộng	Các biến thể thường gặp trong đề thi.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Ghi nhớ	Tóm tắt những điều cốt lõi nhất.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
BÀI TOÁN	Đề bài chính thức.
Ý TƯỞNG	Ý tưởng then chốt của bài học.
KHUÔN MẪU	Đoạn chương trình mẫu dùng lại được cho nhiều bài.
CẮT NHÁNH	Một điều kiện cho phép bỏ qua cả một nhánh của cây tìm kiếm.
ĐẾM SỐ PHÉP TÍNH	Phân tích số trạng thái, số lời gọi, độ phức tạp.
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
VÌ SAO ĐÚNG?	Lập luận chứng minh tính đúng đắn, nhất là chứng minh rằng việc cắt nhánh không làm mất lời giải.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Trong các hình vẽ, màu sắc cũng có ý nghĩa cố định: nút **vàng** là trạng thái đang xét; nút **xanh lá** là trạng thái hợp lệ, đang đi tiếp; nút **xanh dương** là trạng thái đã xét xong hoặc lấy lại từ bảng nhớ; nút **trắng** là trạng thái chưa xét; nút **tím** là điểm dừng (trường hợp cơ sở hoặc một lời giải hoàn chỉnh); nút **đỏ** hay dấu × là nhánh **bị cắt**.

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Với mỗi bài toán quay lui, hãy tự vẽ vài tầng đầu tiên của cây tìm kiếm trước khi xem hình, và tự trả lời bốn câu hỏi: **trạng thái** là gì, ở mỗi bước có những **lựa chọn** nào, khi nào một trạng thái là **lời giải hoàn chỉnh**, và khi nào có thể **cắt** một nhánh. Chỉ khi tự viết lại được chương trình mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **3**

CHƯƠNG 1. ĐỆ QUY **6**

Bài 1. Đệ quy: hàm tự gọi chính nó 6
Bài 2. Sinh dãy nhị phân và tập con 18
Bài 3. Sinh hoán vị, chỉnh hợp và tổ hợp 27

CHƯƠNG 2. QUAY LUI **20**

Bài 4. Mô hình quay lui: Bài toán n quân hậu 35
Bài 5. Quay lui trên lưới: Mê cung và đường đi 43
Bài 6. Quay lui với nhiều ràng buộc: Phân tích số và Sudoku 52

CHƯƠNG 3. NHÁNH CẬN **15**

Bài 7. Cắt tỉa nhánh không khả thi 59
Bài 8. Nhánh cận cho bài toán tối ưu: Người du lịch 68
Bài 9. Bài toán cái túi và cận phân số 76

CHƯƠNG 4. KĨ THUẬT NÂNG CAO **20**

Bài 10. Mặt nạ bit và thứ tự duyệt 83
Bài 11. Quay lui có ghi nhớ: Từ quay lui đến quy hoạch động 91
Bài 12. Tìm kiếm sâu dần và IDA* 99

PHỤ LỤC: BẢNG TRA CỨU NHANH **107**

Lê Thanh Nam

CHƯƠNG 1

ĐỆ QUY

Trước khi học cách "thử mọi khả năng", ta cần một công cụ để diễn đạt việc thử ấy một cách gọn gàng: Đệ quy. Chương này bắt đầu từ câu hỏi đơn giản nhất (một hàm tự gọi chính nó thì máy tính làm gì?), đi qua hiện tượng bùng nổ lời gọi và cách chặn nó bằng ghi nhớ, rồi dùng đệ quy để liệt kê mọi dãy nhị phân, mọi tập con, mọi hoán vị và tổ hợp. Cuối chương, em sẽ có trong tay khuôn mẫu "thử từng lựa chọn cho từng vị trí" mà cả cuốn sách dựa vào.

■ TRONG CHƯƠNG NÀY

- **Bài 1.** Đệ quy: hàm tự gọi chính nó
- **Bài 2.** Sinh dãy nhị phân và tập con
- **Bài 3.** Sinh hoán vị, chỉnh hợp và tổ hợp

BÀI 1 Đệ quy: hàm tự gọi chính nó

Học xong bài này, em sẽ:

✓ Hiểu một hàm đệ quy gồm điểm dừng và bước đệ quy, và biết máy tính thực hiện nó bằng ngăn xếp lời gọi.

✓ Vẽ được cây đệ quy và đếm được số lời gọi; thấy tận mắt vì sao đệ quy "ngây thơ" có thể bùng nổ theo cấp số nhân.

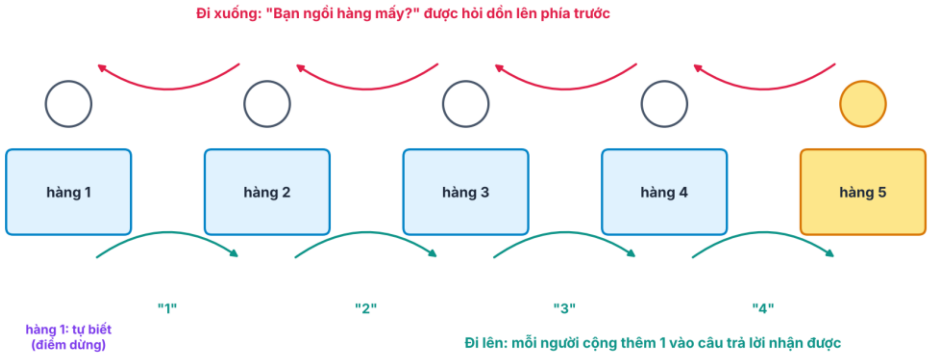
✓ Dùng đệ quy có nhớ để mỗi bài toán con chỉ được tính một lần.

✓ Biết giới hạn độ sâu đệ quy của máy chấm và cách tránh lỗi tràn ngăn xếp.

✓ Thiết kế hàm đệ quy: Bài toán con là gì, điểm dừng ở đâu, ghép kết quả thế nào.

■ KHỞI ĐỘNG

Em đang ngồi trong một rạp chiếu phim tối om và muốn biết mình ngồi ở hàng thứ mấy. Không nhìn thấy gì, em hỏi người ngồi ngay phía trước: "Bạn ngồi hàng mấy?". Người đó cũng không biết, nên lại hỏi người phía trước mình, và cứ thế. Cuối cùng câu hỏi đến người ngồi **hàng đầu tiên**: người này biết ngay mình ở hàng 1 và trả lời. Câu trả lời được chuyền ngược về phía sau, mỗi người cộng thêm 1. Không ai phải đếm cả rạp, mỗi người chỉ làm một việc rất nhỏ. Đó chính là **đệ quy**.



Hình 1.1. Câu hỏi chuyền dồn lên phía trước (đi xuống), câu trả lời được chuyền ngược về phía sau (đi lên)

1. Hàm đệ quy đầu tiên

Một hàm được gọi là **đệ quy** nếu trong thân hàm có lời gọi tới **chính nó**. Câu chuyện ở rạp phim có thể viết thành hàm:

‹› Code ý chính: Hàng thứ mấy?

```
int row(int person) {
    if (person == 1) return 1;        // điểm dừng: người ở hàng đầu tự biết
    return row(person - 1) + 1;       // hỏi người phía trước, rồi cộng 1
}
```

Mọi hàm đệ quy đúng đắn đều có hai phần:

> **■ HAI PHẦN CỦA MỘT HÀM ĐỆ QUY**
>
> Điểm dừng (trường hợp cơ sở): một bài toán đủ nhỏ để trả lời ngay, không cần gọi đệ quy. Ở đây: người hàng 1. Bước đệ quy: đưa bài toán về một (hoặc vài) bài toán cùng dạng nhưng nhỏ hơn, gọi chính hàm đó để giải, rồi ghép kết quả. Ở đây: hỏi người phía trước, cộng thêm 1. Mỗi lần gọi đệ quy, bài toán phải tiến gần hơn tới điểm dừng; nếu không, hàm sẽ gọi mãi không dừng.

Một ví dụ quen thuộc khác là giai thừa: $n! = 1 \cdot 2 \cdot \ldots \cdot n$. Vì $n! = (n-1)! \cdot n$ và $0! = 1$, ta viết:

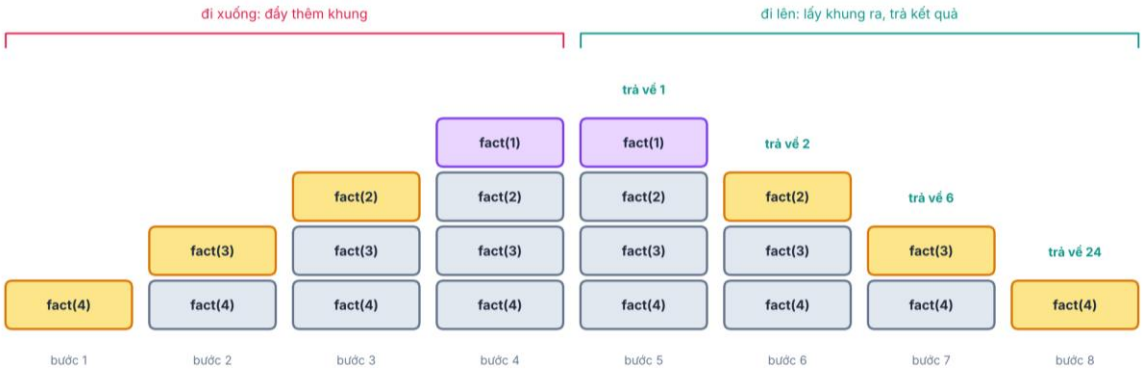
‹› Code ý chính: Giai thừa bằng đệ quy

```
long long fact(int n) {
    if (n == 0) return 1;             // điểm dừng
    return fact(n - 1) * n;           // bước đệ quy
}
```

Máy tính thực hiện đệ quy thế nào: Ngăn xếp lời gọi

Khi fact(4) gọi fact(3), lời gọi fact(4) **chưa xong**: nó còn phải chờ kết quả của fact(3) để nhân với 4. Máy tính ghi nhớ "việc đang làm dở" của fact(4) (giá trị n = 4, chỗ cần quay về) vào một vùng nhớ gọi là **khung** (frame), rồi đặt khung đó lên một chồng gọi là **ngăn xếp lời gọi** (call stack). fact(3) lại đặt thêm một khung, và cứ thế cho tới điểm dừng. (Để hình gọn, Hình 1.2 dừng ở fact(1) = 1 và không vẽ khung của fact(0).)

Lê Thành Nam



Hình 1.2. Ngăn xếp lời gọi của fact(4): bốn bước đầu đẩy thêm khung, bốn bước sau lần lượt lấy khung trên cùng ra và trả kết quả xuống khung bên dưới

Hai giai đoạn trong hình là hai giai đoạn của **mọi** hàm đệ quy:

- **Đi xuống:** các lời gọi lồng nhau, ngăn xếp cao dần. Không phép nhân nào được thực hiện.
- **Đi lên:** từ điểm dừng, mỗi khung nhận kết quả từ khung phía trên, làm nốt phần việc dở dang (nhân với n), trả kết quả xuống khung phía dưới, rồi bị xóa khỏi ngăn xếp.

Chiều cao lớn nhất của ngăn xếp gọi là **độ sâu đệ quy**. Ở fact(n), độ sâu là n + 1. Em sẽ thấy ở mục 4 rằng độ sâu này có giới hạn.

2. Bài toán LEOTHANG

■ BÀI TOÁN: LEOTHANG

Một cầu thang có n bậc. Mỗi bước, em được leo lên **1 bậc** hoặc **2 bậc**. Có bao nhiêu cách leo từ chân cầu thang lên đúng bậc thứ n? In kết quả theo modulo $10^9 + 7$. **Dữ liệu vào:** Một số nguyên n ($1 \le n \le 10^5$). **Kết quả:** Số cách leo, lấy dư cho $10^9 + 7$.

Dữ liệu vào	Kết quả	Giải thích
3	3	1 + 1 + 1, 1 + 2, 2 + 1.
5	8	
50	365010934	Số cách thật là 20 365 011 074.

Tìm công thức đệ quy

Hãy nhìn vào **bước cuối cùng**. Để đứng ở bậc n, bước cuối hoặc là bước 1 bậc (từ bậc n − 1), hoặc là bước 2 bậc (từ bậc n − 2). Hai nhóm cách leo này không trùng nhau (bước cuối khác nhau), và gộp lại thì đủ mọi cách. Gọi ways(n) là số cách leo lên bậc n:

■ CÔNG THỨC

ways(n) = ways(n − 1) + ways(n − 2) với n ≥ 2 Điểm dừng: ways(0) = 1 (đứng yên ở chân cầu thang là một cách), ways(1) = 1.

Đây là dãy Fibonacci: 1, 1, 2, 3, 5, 8, 13, ... Công thức chép thẳng thành chương trình:

Lê Thành Nam

Cách ngây thơ: đệ quy chép đúng công thức, có đếm số lời gọi

```cpp
#include <cstdio>
#include <iostream>
using namespace std;

long long calls = 0;                  // đếm số lời gọi hàm (chỉ để quan sát)

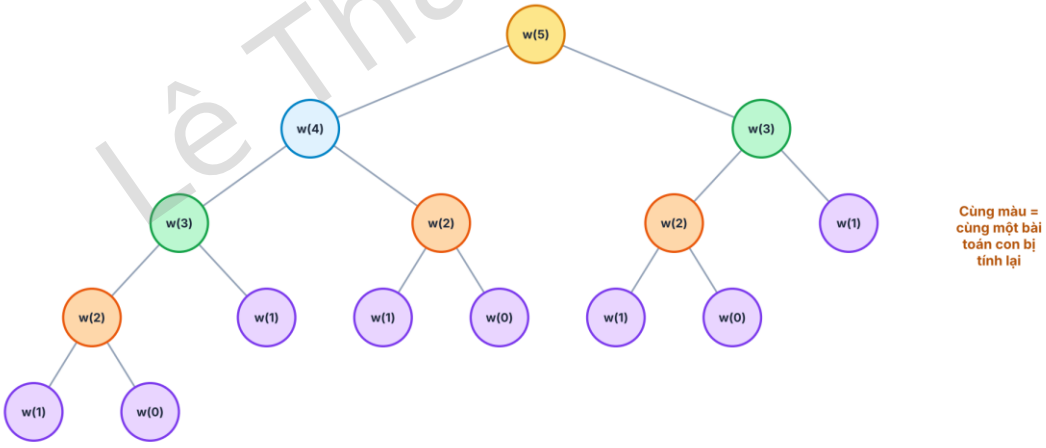
// số cách leo n bậc, mỗi bước 1 hoặc 2 bậc
long long ways(int n) {
    calls++;
    if (n <= 1) return 1;             // điểm dừng: 0 bậc hoặc 1 bậc có đúng 1 cách
    return ways(n - 1) + ways(n - 2);     // bước cuối là 1 bậc, hoặc bước cuối là 2 bậc
}

int main() {
    int n;
    cin >> n;
    long long res = ways(n);
    cout << res << '\n';
    cerr << "so loi goi: " << calls << '\n';
    return 0;
}
```

Chương trình đúng: với n = 5 nó in 8. Biến calls đếm số lần hàm ways được gọi, để ta quan sát.

3. Bế tắc: Cây đệ quy bùng nổ

Hãy vẽ mọi lời gọi khi tính ways(5). Mỗi nút là một lời gọi, các con của nút là các lời gọi mà nó sinh ra:



Hình 1.3. *Cây đệ quy của ways(5): 15 lời gọi. ways(3) được tính 2 lần, ways(2) được tính 3 lần*

Cùng một câu hỏi "có bao nhiêu cách leo lên bậc 2?" được hỏi đi hỏi lại **ba lần**, và mỗi lần chương trình lại tính từ đầu. Với n lớn hơn, sự lãng phí tăng nhanh khủng khiếp. Gọi C(n) là số lời gọi khi tính ways(n): C(0) = C(1) = 1 và C(n) = 1 + C(n − 1) + C(n − 2). Bảng dưới là số lời gọi và thời gian **đo thật** trên máy chấm:

Lê Thành Nam



Hình 1.4. Số lời gọi của đệ quy thường (đỏ) và đệ quy có nhớ (xanh), thang logarit

■ ĐẾM SỐ PHÉP TÍNH

- C(n) thỏa cùng công thức truy hồi với Fibonacci (cộng thêm 1), nên C(n) = 2 · ways(n) − 1 ≈ 2 · $1{,}618^n$: số lời gọi tăng theo cấp số nhân, mỗi khi n tăng 5 thì gấp khoảng 11 lần.
- n = 40: 331 triệu lời gọi, 0,23 giây. n = 45: 3,7 tỉ lời gọi, 2,9 giây. n = 50: khoảng 41 tỉ lời gọi, hơn 30 giây. n = 100: khoảng 10^{21} lời gọi, hàng nghìn năm.
- Trong khi đó, chỉ có n + 1 bài toán con khác nhau: ways(0), ways(1), ..., ways(n).

■ BẾ TẮC

Cây đệ quy có số nút tăng theo cấp số nhân, nhưng số bài toán con khác nhau chỉ là n + 1. Hầu hết công sức bị lãng phí vào việc tính lại những gì đã tính. Làm sao để mỗi bài toán con chỉ được tính một lần?

4. Phá bế tắc: Đệ quy có nhớ

Bước 1 Ghi lại kết quả vào một bảng

Dùng mảng memo với memo[i] = −1 nghĩa là "ways(i) chưa được tính". Mỗi khi tính xong ways(i), ta **ghi** kết quả vào memo[i]. Mỗi khi được hỏi ways(i), ta **xem bảng trước**: nếu đã có thì trả về ngay, không gọi đệ quy nữa.

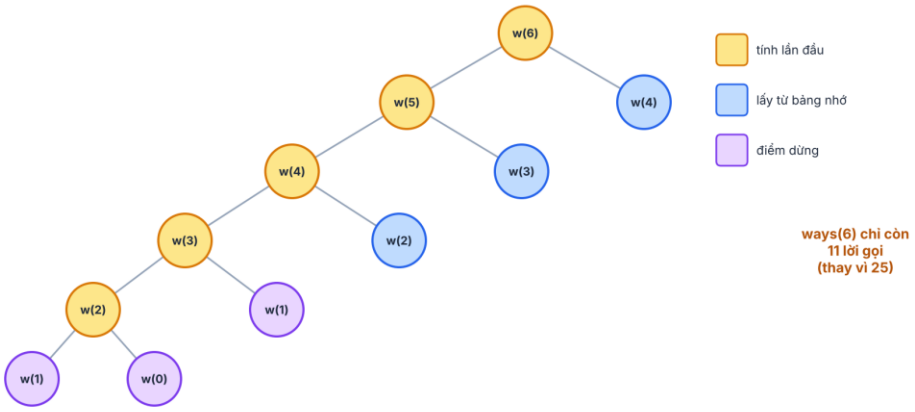
■ ĐỆ QUY CÓ NHỚ (MEMOIZATION)

f(x): nếu x là điểm dừng thì trả về đáp án; nếu memo[x] đã có thì trả về memo[x]; ngược lại tính bằng các lời gọi đệ quy, ghi vào memo[x], rồi trả về.

Bước 2 Cây đệ quy bị "tỉa" gần hết

Với bảng nhớ, lời gọi ways(i) chỉ thật sự gọi đệ quy ở **lần đầu tiên**. Các lần sau, nó trả về ngay từ bảng:

Lê Thanh Nam



Hình 1.5. Cây đệ quy có nhớ của ways(6): các nút xanh dương trả về ngay từ bảng, không sinh lời gọi con

Mỗi ways(i) với $2 \le i \le n$ được tính đúng một lần và gọi hai lời gọi con, nên tổng cộng chỉ có **2n − 1 lời gọi**: O(n). Với n = 45, từ 3,7 tỉ lời gọi xuống còn 89.

Bước 3 Chương trình hoàn chỉnh

Đáp án lớn rất nhanh (ways(50) đã vượt $2 \cdot 10^{10}$), nên ta lấy dư cho $10^9 + 7$ ngay khi cộng.

LEOTHANG.CPP: Đệ quy có nhớ, O(n)

```
1  #include <cstdio>
2  #include <iostream>
3  #include <vector>
4  using namespace std;
5
6  const long long MOD = 1000000007;
7  vector<long long> memo;          // memo[i] = -1 nghĩa là chưa tính
8
9  long long ways(int n) {
10     if (n <= 1) return 1;              // điểm dừng
11     if (memo[n] != -1) return memo[n];   // đã tính rồi: dùng lại
12     memo[n] = (ways(n - 1) + ways(n - 2)) % MOD;
13     return memo[n];
14 }
15
16 int main() {
17     ios_base::sync_with_stdio(false);
18     cin.tie(nullptr);
19     if (fopen("LEOTHANG.INP", "r")) {
20         freopen("LEOTHANG.INP", "r", stdin);
21         freopen("LEOTHANG.OUT", "w", stdout);
22     }
23     int n;
24     cin >> n;
25     memo.assign(n + 1, -1);
26     cout << ways(n) << '\n';
27     return 0;
28 }
```

- Dòng 11 là toàn bộ "phép màu": nếu đã tính rồi thì trả về ngay.
- Dòng 25 khởi tạo bảng nhớ bằng −1. Không thể dùng 0 để đánh dấu "chưa tính", vì một kết quả sau khi lấy dư hoàn toàn có thể bằng 0.

Lê Thành Nam

Bước 4 Giới hạn độ sâu đệ quy

Gọi ways(100000) sẽ tạo ra một chuỗi lời gọi lồng nhau ways(100000) → ways(99999) → ... → ways(1): độ sâu 10^5. Mỗi khung trên ngăn xếp tốn vài chục byte, và ngăn xếp có kích thước **giới hạn**. Khi vượt quá, chương trình bị dừng đột ngột với lỗi **tràn ngăn xếp** (stack overflow), máy chấm báo "Runtime Error".

▪ ĐO THẬT VỚI HÀM WAYS

- Ngăn xếp 1 MB (mặc định của các chương trình biên dịch trên **Windows**): chạy được tới độ sâu khoảng **31 000**. n = 10^5 bị tràn ngăn xếp.
- Ngăn xếp 8 MB (mặc định trên Linux): chạy được tới độ sâu khoảng **260 000**.
- Hàm có nhiều biến cục bộ hơn thì mỗi khung lớn hơn, độ sâu tối đa còn nhỏ hơn.

Có ba cách xử lí:

1. Tăng kích thước ngăn xếp khi biên dịch, ví dụ thêm tùy chọn -Wl,--stack=268435456 (256 MB) trên Windows. Nhiều kì thi cấu hình sẵn điều này, nhưng em phải hỏi ban tổ chức, không được đoán.

2. Gọi trước các bài toán nhỏ: trước khi gọi ways(n), gọi ways(1000), ways(2000), ... theo thứ tự tăng dần; mỗi lời gọi chỉ sâu thêm khoảng 1000 tầng vì phần bên dưới đã có trong bảng.

3. Bỏ đệ quy, tính từ dưới lên bằng vòng lặp: f[0] = f[1] = 1, rồi lần lượt f[2], f[3], ..., f[n]. Đây là cách an toàn nhất, và chính là quy hoạch động.

LEOTHANG.CPP: Tính từ dưới lên, không đệ quy

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

const long long MOD = 1000000007;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("LEOTHANG.INP", "r")) {
        freopen("LEOTHANG.INP", "r", stdin);
        freopen("LEOTHANG.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    vector<long long> f(n + 2);
    f[0] = 1; f[1] = 1;              // hai điểm dừng
    for (int i = 2; i <= n; i++) f[i] = (f[i - 1] + f[i - 2]) % MOD;   // tính từ dưới lên
    cout << f[n] << '\n';
    return 0;
}
```

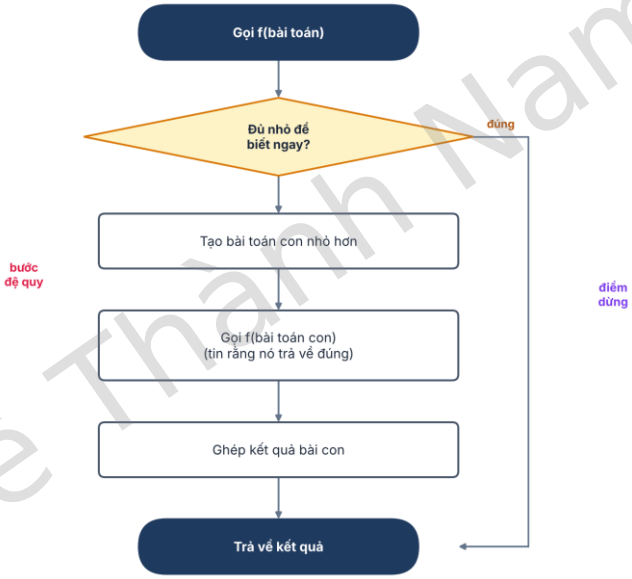
Lê Thành Nam

■ ĐỆ QUY CÓ NHỚ VÀ QUY HOẠCH ĐỘNG

Đệ quy có nhớ là quy hoạch động viết "từ trên xuống": ta đặt câu hỏi lớn trước, và chỉ những bài toán con **thật sự cần** mới được tính. Cách lặp là quy hoạch động "từ dưới lên". Hai cách cho cùng kết quả và cùng độ phức tạp; đệ quy có nhớ dễ viết hơn khi thứ tự tính các bài toán con phức tạp (em sẽ gặp lại ở Bài 11), còn cách lặp không lo tràn ngăn xếp.

5. Thiết kế một hàm đệ quy

Điều khó nhất khi mới học đệ quy là cảm giác "làm sao hàm biết được kết quả của chính nó khi nó còn chưa viết xong?". Bí quyết là **niềm tin đệ quy**: khi viết bước đệ quy, hãy **tin** rằng lời gọi với bài toán nhỏ hơn trả về đúng, và chỉ lo một việc: **từ kết quả đúng của bài nhỏ, làm sao ghép ra kết quả của bài lớn?** Nếu điểm dừng đúng và bước ghép đúng thì hàm đúng (đó chính là phép chứng minh **quy nạp**).



Hình 1.6. Sơ đồ chung của một hàm đệ quy

■ BA CÂU HỎI KHI VIẾT HÀM ĐỆ QUY

1. Bài toán con là gì? Phát biểu thành một câu đầy đủ, kể cả các tham số. Ví dụ: "palin(l, r) trả về true nếu xâu con s[l..r] đối xứng".

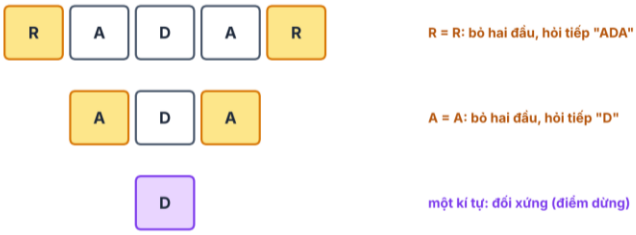
2. Điểm dừng ở đâu? Bài toán nhỏ nhất nào biết ngay đáp án? Kiểm tra rằng **mọi** chuỗi lời gọi đều chạm tới nó.

3. Ghép thế nào? Tin rằng bài con trả về đúng; từ đó suy ra đáp án bài lớn.

Ví dụ: Kiểm tra xâu đối xứng

Xâu s đối xứng nếu đọc xuôi và đọc ngược như nhau (RADAR, ABBA). Bài toán con: xâu con s[l..r] có đối xứng không? Điểm dừng: xâu con rỗng hoặc chỉ một kí tự ($l \geq r$) luôn đối xứng. Ghép: s[l..r] đối xứng khi và chỉ khi hai đầu bằng nhau và phần bên trong s[l + 1..r − 1] đối xứng.

Lê Thành Nam



Hình 1.7. Kiểm tra RADAR: mỗi lời gọi bỏ đi hai đầu bằng nhau

DOIXUNG.CPP: Kiểm tra xâu đối xứng bằng đệ quy, O(n)

```cpp
#include <cstdio>
#include <iostream>
#include <string>
using namespace std;

string s;

// xâu con s[l..r] có đối xứng không?
bool palin(int l, int r) {
    if (l >= r) return true;              // điểm dừng: xâu rỗng hoặc một kí tự
    if (s[l] != s[r]) return false;       // hai đầu khác nhau: chắc chắn không đối xứng
    return palin(l + 1, r - 1);           // bỏ hai đầu, hỏi phần bên trong
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("DOIXUNG.INP", "r")) {
        freopen("DOIXUNG.INP", "r", stdin);
        freopen("DOIXUNG.OUT", "w", stdout);
    }
    cin >> s;
    cout << (palin(0, (int)s.size() - 1) ? "YES" : "NO") << '\n';
    return 0;
}
```

Lê Thanh Nam

Hàm palin nhận chỉ số l, r thay vì một xâu con mới. Nếu viết palin(s.substr(1, n − 2)), mỗi lời gọi phải sao chép gần như cả xâu, độ phức tạp thành $O(n^2)$.

Ví dụ: In một số ở hệ cơ số b

Muốn in n ở hệ cơ số b ($2 \le b \le 10$): chữ số cuối cùng là n mod b, các chữ số phía trước chính là cách viết của $\lfloor n/b \rfloor$. Vì chữ số cuối phải in sau cùng, ta gọi đệ quy trước rồi mới in:

Code ý chính: In n ở hệ cơ số b

```cpp
void printBase(long long n, int b) {
    if (n >= b) printBase(n / b, b);    // in các chữ số phía trước (nếu có)
    cout << n % b;                      // rồi mới in chữ số cuối
}
```

Đổi thứ tự hai dòng (in trước, gọi sau) sẽ in các chữ số theo thứ tự ngược. Vị trí của lời gọi đệ quy so với phần việc còn lại quyết định thứ tự thực hiện: phần việc đặt trước lời gọi được làm ở giai đoạn đi xuống, phần việc đặt sau lời gọi được làm ở giai đoạn đi lên.

6. Mở rộng

Đệ quy nhiều nhánh: Một lời gọi sinh ra nhiều lời gọi

Hàm ways gọi chính nó hai lần, nên cây đệ quy là cây nhị phân. Khi một hàm gọi chính nó k lần với bài toán nhỏ hơn 1 đơn vị, cây có tới k^n lá. Ở Bài 2 và Bài 3, ta sẽ cố ý tạo ra những cây như vậy để liệt kê mọi khả năng: mỗi lá là một cấu hình (một dãy nhị phân, một hoán vị). Khi đó số lá lớn là tất yếu vì đề bài yêu cầu liệt kê tất cả; còn ở bài LEOTHANG, số lá lớn là lãng phí vì các lá trùng nhau. Phân biệt hai tình huống này là kĩ năng quan trọng: thấy bài toán con lặp lại thì nghĩ tới ghi nhớ; các nhánh khác nhau thật sự thì nghĩ tới cắt nhánh (Chương 3).

Đệ quy đi xuống theo cấu trúc

Nhiều đối tượng được định nghĩa đệ quy: một biểu thức số học là các "hạng tử" nối bằng + và −, mỗi hạng tử là các "thừa số" nối bằng ×, mỗi thừa số là một số hoặc một biểu thức đặt trong ngoặc. Một bộ phân tích viết theo đúng định nghĩa này (mỗi khái niệm một hàm, các hàm gọi lẫn nhau) tính được mọi biểu thức. Bài tập 1.2 (BIỂU THỨC) hướng dẫn cách viết.

Đệ quy giảm nhanh: Bài toán Josephus

n người đứng thành vòng tròn, cứ đếm đến người thứ k thì người đó bị loại. Đánh số từ 0, gọi J(n) là vị trí người cuối cùng còn lại. Sau khi người thứ k bị loại, đánh số lại từ người kế tiếp, ta được bài toán cùng dạng với n − 1 người, nên J(n) = (J(n − 1) + k) mod n. Công thức này cho lời giải O(n). Khi n tới 10^{18}, ta cần một bước đệ quy giảm n nhanh hơn (loại cả một vòng ⌊n/k⌋ người một lúc): đó là nội dung bài tập 1.3 (VÒNG TRÒN).

■ BẪY THƯỜNG GẶP

- Thiếu điểm dừng hoặc điểm dừng không bao giờ chạm tới: ví dụ ways(n) với n âm khi chỉ kiểm tra `n == 0`: hàm gọi mãi xuống −1, −2, ... cho tới khi tràn ngăn xếp. Viết điều kiện dừng bao quát (`n <= 1`).
- Đánh dấu "chưa tính" bằng 0 trong bảng nhớ, trong khi 0 là một kết quả hợp lệ (sau khi lấy dư): bảng nhớ mất tác dụng ở đúng những ô đó.
- Tràn ngăn xếp với độ sâu lớn, đặc biệt khi chấm trên Windows (1 MB): độ sâu trên vài chục nghìn là nguy hiểm.
- Truyền vector hoặc xâu theo giá trị (`void f(vector<int> a)`): mỗi lời gọi sao chép toàn bộ, chậm đi rất nhiều. Truyền tham chiếu (`vector<int>& a`) hoặc dùng biến toàn cục và chỉ số.
- Gọi đệ quy hai lần với cùng tham số (như `f(n − 1) + f(n − 1)`) thay vì gọi một lần và lưu vào biến.

Lê Thanh Nam

■ GHI NHỚ

- Hàm đệ quy = **điểm dừng** + **bước đệ quy** đưa về bài toán nhỏ hơn cùng dạng.
- Máy tính thực hiện đệ quy bằng **ngăn xếp lời gọi**: đi xuống (đẩy khung), đi lên (lấy khung, trả kết quả). Độ sâu đệ quy bị giới hạn bởi kích thước ngăn xếp.
- Vẽ **cây đệ quy** để đếm số lời gọi. Nếu cùng một bài toán con xuất hiện nhiều lần, dùng **đệ quy có nhớ**: mỗi bài con chỉ tính một lần.
- Khi viết bước đệ quy, hãy **tin** bài con trả về đúng và chỉ lo phần ghép.

Luyện tập

Bài	Tên	Gợi ý liên hệ
1.1	BẬC THANG	Mở rộng LEOTHANG: mỗi bước leo 1 đến k bậc, có bậc bị hỏng. Đệ quy có nhớ, chú ý độ phức tạp O(n · k).
1.2	BIỂU THỨC	Tính giá trị biểu thức có ngoặc bằng các hàm đệ quy gọi lẫn nhau.
1.3	VÒNG TRÒN	Bài toán Josephus với n tới 10^{18}: đệ quy giảm n theo cấp số nhân.

Tự kiểm tra nhanh:

1. Hàm sau in ra gì khi gọi f(3)? `void f(int n) { if (n == 0) return; cout << n; f(n − 1); cout << n; }` *(Đáp số: 321123. Phần in trước lời gọi chạy khi đi xuống, phần in sau chạy khi đi lên.)*

2. Không dùng bảng nhớ, tính ways(6) cần bao nhiêu lời gọi? *(Đáp số: C(6) = 2 · ways(6) − 1 = 2 · 13 − 1 = 25.)*

3. Hàm `int g(int n) { if (n == 1) return 0; return g(n / 2) + 1; }` tính gì, và độ sâu đệ quy với $n = 10^9$ là bao nhiêu? *(Đáp số: ⌊log₂ n⌋; độ sâu 30, hoàn toàn an toàn. Đệ quy giảm theo cấp số nhân không bao giờ lo tràn ngăn xếp.)*

Lê Thành Nam